# Proyecto I: Programación avanzada en Python

Alberto Rey
Máster Universitario en Inteligencia Artificial Aplicada (ICAI) - DSIA
13 de octubre de 2026
Repositorio: https://github.com/albertoreydlf/dsia-26-27-reydelafe-alberto

## 1. Qué he hecho

En este proyecto he hecho dos programas pequeños en Python que hacen lo mismo pero con datos distintos. Cada programa carga un CSV, comprueba qué filas son correctas y cuáles no, calcula unas métricas con pandas y se puede ejecutar desde la terminal. Además, a cada uno le he escrito tests con pytest y he configurado un CI en GitHub Actions para que los tests se pasen solos cada vez que subo cambios.

- Parte A: `ventas_app`, con el CSV de ventas que usamos en el curso (150 filas).
- Parte B: `internet_app`, con un dataset de Kaggle sobre sueño y uso del móvil antes de dormir (8.500 filas).

La parte B la hice siguiendo la misma forma que la A. Así los dos programas se parecen mucho y, si entiendo uno, entiendo el otro.

## 2. Cómo lo he organizado

### 2.1 Carpetas

Todo el proyecto está dentro de la carpeta `proyecto/` del repositorio. Lo he separado de los ejercicios del curso (el E3 está aparte, en `2_pruebas_y_despliegue/ejercicios/E3_solucion`) para no mezclar lo que es ejercicio con lo que es entrega del proyecto.

```
proyecto/
  ventas_app/       (Parte A: loader.py, validator.py, metrics.py, cli.py)
  internet_app/     (Parte B: loader.py, validator.py, metrics.py, cli.py)
  tests/ventas/     (tests de la Parte A)
  tests/internet/   (tests de la Parte B)
  data/             (los dos CSV)
  pytest.ini

.github/workflows/ci.yml   (en la raíz del repositorio)
```

### 2.2 Cuatro ficheros por programa

Cada programa tiene los mismos cuatro ficheros y cada uno hace una sola cosa:

- `loader.py`: lee el CSV y avisa si algo va mal (el fichero no existe, faltan columnas).
- `validator.py`: separa las filas válidas de las inválidas aplicando unas reglas.
- `metrics.py`: calcula las métricas sobre las filas válidas.
- `cli.py`: junta todo y es lo que se ejecuta desde la terminal.

Lo hice así porque si algo falla sé en qué fichero tengo que mirar, y puedo probar cada pieza por separado en los tests. El recorrido de los datos es siempre el mismo: CSV, `load`, `validar`,

métricas, y al final `cli` guarda un CSV con las filas válidas e imprime el resumen.

## 3. Parte A: ventas_app

Es el programa del ejercicio de ventas. Lo copié dentro de `proyecto/` para que el proyecto sea autónomo. Los datos están en `proyecto/data/ventas.csv` y tienen estas columnas: fecha, region, producto, unidades, precio_unitario y cliente_id. En el código que enseño en este documento, alguna línea muy larga sale partida en dos para que quepa en la página; en el fichero real es una sola línea.

### 3.1 loader.py

*proyecto/ventas_app/loader.py*

```
import pandas as pd
from pathlib import Path


class DataLoadError(Exception):
    """Error al cargar datos de origen."""

def load(path:Path) -> pd.DataFrame:
    if not path.exists():
        raise DataLoadError(f"No existe el fichero {path}")
    df = pd.read_csv(path)
    return df
```

`DataLoadError` es un error que me defino yo, heredando de `Exception`. Lo uso para que, cuando falla la carga, el error diga claramente qué ha pasado ("No existe el fichero ...") y, además, en los tests pueda comprobar que sale justo este error y no otro.

En `load` primero compruebo con `path.exists()` que el fichero existe. Si no existe lanzo mi error. Si existe, lo leo con `pd.read_csv` y devuelvo el DataFrame.

### 3.2 validator.py

*proyecto/ventas_app/validator.py*

```
import pandas as pd


class ValidationError(Exception):
    """Datos que no cumplen reglas de negocio."""


def validar_ventas(frame: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    copia_datos = frame.copy() #lo usamos para trabajar sobre una copia
    copia_datos["unidades"] = pd.to_numeric(copia_datos["unidades"], errors = "coerce")
    copia_datos["precio_unitario"] = pd.to_numeric(copia_datos["precio_unitario"], errors =
        "coerce")

    # regla: unidades y precio existen y son mayores que 0
    ok = (
        copia_datos["unidades"].notna()
        & (copia_datos["unidades"] > 0)
        & copia_datos["precio_unitario"].notna()
        & (copia_datos["precio_unitario"] > 0)
    )

    validos = copia_datos.loc[ok].copy()        #cuando se cumpla ok
    errores = copia_datos.loc[~ok].copy()       #~ok es cuando NO se cumpla ok
    validos["importe"] = validos["unidades"] * validos["precio_unitario"]
    return validos, errores
```

`validar_ventas` recibe el DataFrame y devuelve dos: el de las filas válidas y el de las inválidas.

- `frame.copy()`: trabajo sobre una copia para no modificar el DataFrame original.
- `pd.to_numeric(..., errors="coerce")`: convierte la columna a número. Si hay algo que no se puede convertir (por ejemplo un texto), en vez de dar error lo deja como NaN, y esa fila acabará siendo inválida.
- `ok` es una máscara de verdadero/falso, una por fila. Una fila es válida si unidades y precio existen y son mayores que 0. Uso `&` (y no `and`) porque estoy combinando columnas enteras de pandas.
- `copia_datos.loc[ok]` se queda con las filas donde `ok` es verdadero, y `~ok` es lo contrario (la ~ niega), así que `loc[~ok]` son las inválidas.
- Al final añado la columna `importe = unidades * precio_unitario`, solo para las válidas.

Nota: en este fichero tengo definida una clase `ValidationError` que al final no uso. La dejé porque venía del ejercicio, pero el programa funciona sin ella.

### 3.3 metrics.py

*proyecto/ventas_app/metrics.py*

```
import pandas as pd


def total_por_region(validos: pd.DataFrame):
    return validos.groupby("region")["importe"].sum().sort_values(ascending=False)

TOP_N = 3 #numero de productos que queremos mostrar
def top3_prod_porimporte(validos: pd.DataFrame):
    return validos.groupby("producto")["importe"].sum().sort_values(ascending=False).head(TOP_N)
```

Las dos funciones usan `groupby`, que agrupa las filas por una columna (la región o el producto) y luego se suma el importe de cada grupo. `sort_values(ascending=False)` ordena de mayor a menor. En la segunda, `head(TOP_N)` se queda solo con los 3 primeros. He puesto el 3 en una constante `TOP_N` para no tener un número suelto en medio del código.

Las dos devuelven una Series de pandas, que es como una tabla de una sola columna donde cada valor tiene una etiqueta (aquí, la región o el producto).

### 3.4 cli.py

*proyecto/ventas_app/cli.py*

```
import argparse
from pathlib import Path

from ventas_app.loader import load
from ventas_app.validator import validar_ventas
from ventas_app.metrics import total_por_region

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", required=True)
    parser.add_argument("--output", required=True)
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    frame = load(Path(args.input))  #llamamos a la funcionn que cree en loader(carga el csv)
    validos, errores = validar_ventas(frame)
```

```
    totales = total_por_region(validos)
    validos.to_csv(Path(args.output), index=False)
    print(f"Hay {len(validos)} válidos y {len(errores)} invalidos")
    print(totales)


if __name__ == "__main__":
    main()
```

`argparse` es lo que permite escribir argumentos en la terminal. Aquí defino `--input` (el CSV de entrada) y `--output` (dónde guardar el CSV limpio), los dos obligatorios. En `main` hago los pasos en orden: cargo, valido, calculo el total por región, guardo las filas válidas con `to_csv(..., index=False)` (sin la columna del índice) e imprimo el resumen.

El `if __name__ == "__main__":` del final hace que `main()` solo se ejecute cuando lanzo el fichero directamente, y no cuando otro fichero (como los tests) lo importa.

Se ejecuta desde la carpeta `proyecto/` con:

```
python -m ventas_app.cli --input data/ventas.csv --output /tmp/ventas_limpias.csv
```

Uso `python -m` para que Python encuentre el paquete `ventas_app`, porque dentro del código importo con `from ventas_app.loader import load`.

### 3.5 Resultado

```
Hay 140 válidos y 10 invalidos
region
Este     33122.53
Norte    22914.63
Sur      21396.35
Oeste    19164.66
Name: importe, dtype: float64
```

De las 150 filas, 140 son válidas y 10 no. La región con más ventas es Este.

## 4. Parte B: internet_app

### 4.1 El dataset

- Nombre: Sleep Debt and Screen Time: Late Night Phone Habits
- Origen: https://www.kaggle.com/datasets/samartalwar/sleep-debt-and-screen-time-late-night-phone-habits
- Autor: samartalwar (Kaggle)
- Licencia: CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/)
- Fecha de descarga: 11 de octubre de 2026
- Tiene 8.500 filas y 18 columnas. No lo he modificado.

La licencia CC BY-SA 4.0 me deja usar y compartir los datos si cito al autor, enlazo la licencia e indico si he hecho cambios. Por eso todo eso está en el README del proyecto. Como no he tocado el fichero, no hay cambios que indicar.

Elegí este dataset porque habla de algo que se puede medir con números (minutos de móvil antes de dormir, horas de sueño, fatiga al día siguiente), y eso permite calcular métricas con sentido. Las columnas que uso en las reglas y las métricas son: user_id, age, bedtime_phone_minutes, primary_bedtime_app, screen_brightness_pct, blue_light_filter_active, total_sleep_hours,

deep_sleep_pct, rem_sleep_pct, next_day_fatigue_score y sleep_debt_category.

### 4.2 loader.py

*proyecto/internet_app/loader.py*

```
import pandas as pd
from pathlib import Path

# Columnas que debe tener el CSV para poder trabajar con él
COLUMNAS_REQUERIDAS = [
    "user_id", "age", "gender", "occupation_type", "chronotype",
    "bedtime_phone_minutes", "primary_bedtime_app", "screen_brightness_pct",
    "blue_light_filter_active", "caffeine_post_5pm_mg", "physical_activity_min",
    "sleep_latency_min", "total_sleep_hours", "deep_sleep_pct", "rem_sleep_pct",
    "morning_alarm_snoozes", "next_day_fatigue_score", "sleep_debt_category",
]


class DataLoadError(Exception):
    """Error al cargar los datos de origen."""


def load(path: Path) -> pd.DataFrame:
    """Lee el CSV y comprueba que existen todas las columnas necesarias."""
    if not path.exists():
        raise DataLoadError(f"No existe el fichero {path}")
    df = pd.read_csv(path)
    # Buscamos qué columnas necesarias no están en el CSV
    faltan = []
    for columna in COLUMNAS_REQUERIDAS:
        if columna not in df.columns:
            faltan.append(columna)

    # Si hay alguna que falta, paramos con un error claro
    if len(faltan) > 0:
        raise DataLoadError(f"Faltan columnas en el CSV: {faltan}")
    return df
```

Es parecido al de ventas, con una comprobación más. `COLUMNAS_REQUERIDAS` es la lista de columnas que tiene que tener el CSV. Después de leerlo, recorro esa lista y apunto en `faltan` las que no están en `df.columns`. Si `faltan` no está vacía, lanzo `DataLoadError` diciendo cuáles faltan. Lo hago aquí porque es mejor parar con un mensaje claro que dejar que el programa falle más adelante con un error raro de pandas (un `KeyError`).

### 4.3 validator.py

*proyecto/internet_app/validator.py*

```
import pandas as pd

CATEGORIAS_VALIDAS = ["Optimal Recovery", "Mild Deficit", "Moderate Debt", "Severe Sleep Debt"]
UMBRAL_MOVIL_ALTO = 60  # minutos de móvil antes de dormir a partir de los cuales el uso es
    "alto"

# Columnas que deben ser números
COLUMNAS_NUMERICAS = [
    "age", "bedtime_phone_minutes", "screen_brightness_pct", "blue_light_filter_active",
    "total_sleep_hours", "deep_sleep_pct", "rem_sleep_pct", "next_day_fatigue_score",
]


def validar_sueno(frame: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    copia = frame.copy()  # trabajamos sobre una copia para no modificar el original
    # Si un valor no se puede convertir a número, pasa a NaN (y la fila será inválida)
    for col in COLUMNAS_NUMERICAS:
        copia[col] = pd.to_numeric(copia[col], errors="coerce")
```

```
    # Reglas: una fila es válida solo si cumple TODAS (los NaN no cumplen ninguna)
    ok = (
        copia["user_id"].notna()
        & ~copia["user_id"].duplicated()                        # sin ids repetidos
        & copia["age"].between(18, 65)                          # edad entre 18 y 65
        & copia["bedtime_phone_minutes"].between(0, 180)        # minutos de móvil entre 0 y 180
        & copia["screen_brightness_pct"].between(0, 100)        # brillo es un porcentaje
        & copia["blue_light_filter_active"].isin([0, 1])        # filtro: 0 (no) o 1 (sí)
        & copia["total_sleep_hours"].between(1, 24)             # horas de sueño razonables
        & ((copia["deep_sleep_pct"] + copia["rem_sleep_pct"]) <= 100)  # los % no pueden pasar
            de 100
        & copia["next_day_fatigue_score"].between(1, 10)        # fatiga de 1 a 10
        & copia["sleep_debt_category"].isin(CATEGORIAS_VALIDAS)  # categoría conocida
    )

    validos = copia.loc[ok].copy()   # filas que cumplen todas las reglas
    errores = copia.loc[~ok].copy()  # ~ok son las que NO las cumplen
    validos["movil_alto"] = validos["bedtime_phone_minutes"] >= UMBRAL_MOVIL_ALTO
    return validos, errores
```

Es la misma idea que en ventas, pero con más reglas. Primero convierto a número todas las columnas de `COLUMNAS_NUMERICAS` (con `coerce`). Después construyo la máscara `ok`, y una fila es válida solo si cumple todas:

- `user_id` existe (`notna`) y no está repetido. `duplicated()` marca como repetidas todas las apariciones de un id menos la primera, así que con `~duplicated()` me quedo con la primera y mando a errores las demás.
- `between(a, b)` es verdadero si el valor está entre a y b, incluyendo los dos extremos. Lo uso para la edad (18 a 65), los minutos de móvil (0 a 180), el brillo (0 a 100), las horas de sueño (1 a 24) y la fatiga (1 a 10).
- `isin([0, 1])` comprueba que el filtro de luz azul vale 0 o 1. Con `isin(CATEGORIAS_VALIDAS)` compruebo que la categoría de deuda de sueño es una de las cuatro que existen.
- El sueño profundo más el REM no puede pasar de 100 %.

Si un valor es NaN, todas estas comparaciones dan falso, y por eso una fila con un dato vacío o con texto raro siempre acaba en errores.

Al final, a las válidas les añado la columna `movil_alto`, que es verdadera si usan el móvil 60 minutos o más antes de dormir (`UMBRAL_MOVIL_ALTO`). La uso después en las métricas.

Con el dataset real salen las 8.500 filas válidas y 0 inválidas. Por eso los casos inválidos los pruebo en los tests con un DataFrame pequeño que preparo yo.

### 4.4 metrics.py

*proyecto/internet_app/metrics.py*

```
import pandas as pd


def sueno_medio_por_uso_movil(validos: pd.DataFrame):
    """Horas de sueño medias con uso bajo (False) y alto (True) del móvil.Esto devuelve una
        "Series" de pandas """
    """Una Series es una lista de valores donde cada uno tiene una etiqueta (el índice). Se
        puede ver como una tabla de una sola columna"""
    return validos.groupby("movil_alto")["total_sleep_hours"].mean()


def fatiga_media_por_categoria(validos: pd.DataFrame):
    """Fatiga media del día siguiente según la categoría de sleep debt.Esto devuelve una
```

```
        "Series" de pandas"""
    return validos.groupby("sleep_debt_category")["next_day_fatigue_score"].mean().sort_values(as
        cending=False)


def sueno_medio_por_app(validos: pd.DataFrame):
    """Horas de sueño medias según la app que más se usa antes de dormir"""
    return validos.groupby("primary_bedtime_app")["total_sleep_hours"].mean().sort_values()


def correlacion_movil_sueno(validos: pd.DataFrame) -> float:
    """Correlación entre minutos de móvil y horas de sueño (cerca de -1: más móvil, menos
        sueño)"""
    return float(validos["bedtime_phone_minutes"].corr(validos["total_sleep_hours"]))
```

Son cuatro métricas:

- `sueno_medio_por_uso_movil`: horas de sueño medias de los que usan poco el móvil (False) y de los que lo usan mucho (True).
- `fatiga_media_por_categoria`: fatiga media del día siguiente para cada categoría de deuda de sueño, de mayor a menor.
- `sueno_medio_por_app`: horas de sueño medias según la app que más usan antes de dormir, de menor a mayor.
- `correlacion_movil_sueno`: la correlación entre los minutos de móvil y las horas de sueño. Va de -1 a 1; cerca de -1 quiere decir que a más móvil, menos sueño. Lo convierto con `float()` para devolver un número normal y no un tipo de numpy.

### 4.5 cli.py

*proyecto/internet_app/cli.py*

```
import argparse
from pathlib import Path

from internet_app.loader import load
from internet_app.validator import validar_sueno
from internet_app.metrics import (
    correlacion_movil_sueno,
    fatiga_media_por_categoria,
    sueno_medio_por_app,
    sueno_medio_por_uso_movil,
)


def parse_args():
    parser = argparse.ArgumentParser(description="Pipeline de hábitos de sueño y pantalla")
    parser.add_argument("--input", required=True)
    parser.add_argument("--output", required=True)
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    frame = load(Path(args.input))  # carga el CSV con el load de loader.py
    validos, errores = validar_sueno(frame) #usamos el codigo de validator.py
    validos.to_csv(Path(args.output), index=False)
    print(f"Hay {len(validos)} válidos y {len(errores)} inválidos")
    print("\nHoras de sueño medias según uso del móvil (True = 60 min o más):")
    print(sueno_medio_por_uso_movil(validos).round(2))
    print("\nFatiga media según categoría de deuda de sueño:")
    print(fatiga_media_por_categoria(validos).round(2))
    print("\nHoras de sueño medias según la app de antes de dormir:")
    print(sueno_medio_por_app(validos).round(2))
    print(f"\nCorrelación entre minutos de móvil y horas de sueño:
```

```
        {correlacion_movil_sueno(validos):.2f}")


if __name__ == "__main__":
    main()
```

Es como el de ventas, pero imprime las cuatro métricas. Uso `.round(2)` para que salgan solo dos decimales y `:.2f` en la correlación por lo mismo. Se ejecuta así, desde `proyecto/`:

```
python -m internet_app.cli --input data/bedtime_screentime_sleep_debt.csv \
    --output /tmp/sueno_validos.csv
```

### 4.6 Resultado

```
Hay 8500 válidos y 0 inválidos

Horas de sueño medias según uso del móvil (True = 60 min o más):
movil_alto
False    6.82
True     5.47

Fatiga media según categoría de deuda de sueño:
Severe Sleep Debt    9.58
Moderate Debt        4.68
Mild Deficit         1.83
Optimal Recovery     1.08

Horas de sueño medias según la app de antes de dormir:
TikTok / Reels              6.17
Instagram / Reddit          6.21
YouTube                     6.30
Messaging / Chat            6.33
Streaming (Netflix/Hulu)    6.36
News / Reading              6.38

Correlación entre minutos de móvil y horas de sueño: -0.55
```

Lo que veo: los que usan el móvil 60 minutos o más antes de dormir duermen de media 5,47 horas, frente a 6,82 de los que lo usan menos (1,35 horas menos). La correlación es -0,55, negativa y de tamaño medio. Por apps, las diferencias son pequeñas (de 6,17 a 6,38 horas), y la que tiene menos sueño es TikTok / Reels. Esto es una correlación, no demuestra que el móvil sea la causa de dormir menos.

## 5. Tests con pytest

### 5.1 Qué son y por qué

Un test es una función que ejecuta una parte de mi código con datos que conozco y comprueba con `assert` que el resultado es el que espero. Si algún día cambio algo y se rompe, los tests me avisan. Los escribo siguiendo el esquema AAA: preparo los datos (Arrange), ejecuto la función (Act) y compruebo el resultado (Assert).

### 5.2 pytest.ini

*proyecto/pytest.ini*

```
[pytest]
# Dónde están los tests
testpaths = tests
# Para que pytest encuentre el paquete ventas_app (y más adelante internet_app)
pythonpath = .
# Marcas propias: así pytest no avisa de que no las conoce
markers =
```

```
    integration: tests que usan ficheros reales (más lentos)
```

`testpaths` le dice a pytest dónde buscar los tests. `pythonpath = .` hace que encuentre los paquetes `ventas_app` e `internet_app`. Y `markers` registra la marca `integration` que uso en algunos tests, para que pytest no avise de que no la conoce.

### 5.3 Fixtures (conftest.py)

Una fixture es una función con `@pytest.fixture` que prepara datos y se los pasa a los tests que la piden poniéndola como parámetro. Así no repito los mismos datos en cada test. En `conftest.py` están las fixtures compartidas por todos los tests de esa carpeta:

- `ventas_mini`: 3 filas, una válida y dos inválidas (una sin unidades y otra con precio negativo).
- `sueno_mini`: 4 personas, una válida (U1) y tres inválidas: U2 tiene 12 años, U3 tiene un 2 en el filtro de luz azul y U4 tiene una categoría que no existe ("Otra").

### 5.4 Qué comprueba cada test

En total hay 31 tests: 12 de la parte A y 19 de la parte B.

Parte A (ventas_app):

- `test_validator.py` (6): separa 1 válida y 2 inválidas; calcula el importe (2 x 10 = 20); no modifica el DataFrame original; y un test con tres casos para el precio (0 y -1 son inválidos, 1 es válido).
- `test_loader.py` (2): lee un CSV pequeño que creo en una carpeta temporal; y comprueba que un fichero que no existe lanza `DataLoadError`.
- `test_metrics.py` (2): el total por región (Norte = 20 + 40 = 60, y viene primero); y que el top de productos tiene 3 y en orden.
- `test_cli.py` (1): ejecuta `main()` entero con ficheros temporales y comprueba que se crea el CSV de salida y que se imprime "Hay 1 válidos y 2 invalidos".
- `test_integracion.py` (1): con el CSV real, comprueba que salen 140 válidas y 10 inválidas.

Parte B (internet_app):

- `test_validator.py` (10): separa 1 válida y 3 inválidas; añade `movil_alto`; no modifica el original; la edad en los bordes (17 no, 18 sí, 65 sí, 66 no); un valor que no es número es inválido; un id repetido es inválido; y que el sueño profundo más el REM no pase de 100.
- `test_loader.py` (3): lee un CSV; un fichero que no existe lanza error; y si falta la columna `age` lanza error y el mensaje la menciona.
- `test_metrics.py` (4): con seis personas con números fáciles, compruebo a mano las cuatro métricas (por ejemplo la media con móvil bajo es (8 + 7 + 7,5) / 3 = 7,5).
- `test_cli.py` (1): igual que el de ventas, con "Hay 1 válidos y 3 inválidos".
- `test_integracion.py` (1): con el CSV real, 8.500 válidas y 0 inválidas.

### 5.5 Herramientas de pytest que he usado

- `@pytest.mark.parametrize`: ejecuta el mismo test con varios valores (los bordes de la edad o del precio).

- `pytest.raises(Error)`: comprueba que el código lanza el error esperado. Con `match="age"` comprueba además que el mensaje contiene esa palabra.
- `pytest.approx`: para comparar números decimales con un pequeño margen, porque 4,1666... no es igual a 4,17 exactamente.
- `tmp_path`: una carpeta temporal que pytest crea y borra sola, para no ensuciar el repositorio con ficheros de prueba.
- `monkeypatch`: cambia algo solo durante el test. Lo uso para simular lo que escribiría el usuario en la terminal (`sys.argv`) al probar `main()`.
- `capsys`: captura lo que el programa imprime por pantalla para poder comprobarlo.
- `@pytest.mark.integration`: marca los tests que usan los CSV reales; así puedo saltármelos con `pytest -m "not integration"`.

### 5.6 Cómo se ejecutan y qué sale

Desde la carpeta `proyecto/`:

```
pytest -q
pytest -q -m "not integration"
pytest --cov=ventas_app --cov=internet_app --cov-report=term-missing --cov-fail-under=60
```

El primero pasa los 31 tests. El segundo se salta los 2 de integración. El tercero mide la cobertura, que es el porcentaje de líneas de mi código que ejecutan los tests, y falla si baja del 60 %. Mi resultado es: 31 tests pasados y 98 % de cobertura.

## 6. CI con GitHub Actions

CI significa integración continua. Es una máquina de GitHub que, cada vez que subo código, descarga el repositorio, instala lo necesario y ejecuta los tests sola. Si algo falla, lo veo en rojo en GitHub. Así no depende de que yo me acuerde de pasar los tests en mi ordenador.

*.github/workflows/ci.yml*

```
name: dsia-ci

# Cuándo se ejecuta: en cada push a main o a una rama practica/*, y en cada pull request
on:
  push:
    branches: [main, "practica/**"]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      # 1. Descargar el código del repo en la máquina de GitHub
      - name: Checkout
        uses: actions/checkout@v4

      # 2. Instalar Python 3.12 (la misma versión que usamos en local)
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      # 3. Instalar las dependencias del curso
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
```

```
      # 4. Tests del ejercicio E3 (flujo del curso)
      - name: Tests E3 (curso)
        working-directory: 2_pruebas_y_despliegue/ejercicios/E3_solucion
        run: pytest -q

      # 5. Tests del Proyecto I, con el mínimo de cobertura del 60 %
      - name: Tests Proyecto I + cobertura (>= 60%)
        working-directory: proyecto
        run: |
          pytest -q \
            --cov=ventas_app \
            --cov=internet_app \
            --cov-report=term-missing \
            --cov-fail-under=60
```

- `on:` dice cuándo se ejecuta: en cada push a `main` o a una rama `practica/...`, y en cada pull request.
- `runs-on: ubuntu-latest` es la máquina virtual (Linux) donde se ejecuta todo.
- `uses:` usa una acción ya hecha por otros (un repositorio público de GitHub). `actions/checkout` descarga mi código y `actions/setup-python` instala Python 3.12.
- `run:` ejecuta comandos de terminal, como los que escribiría yo: instalar el `requirements.txt` y lanzar pytest.
- `working-directory` es la carpeta desde la que se lanza cada comando. Pasa primero los tests del E3 y después los del proyecto, con `--cov-fail-under=60` para que falle si la cobertura baja del 60 %.

Las ejecuciones se ven en https://github.com/albertoreydlf/dsia-26-27-reydelafe-alberto/actions/workflows/ci.yml. Las que he hecho están en verde.

## 7. Git y entrega

He trabajado en una rama para esta sesión, `practica/sesion-04`, y no directamente en `main`. Los commits los he escrito en español y los he ido haciendo por partes (primero la estructura y los tests de la parte A, después el CI, después la parte B con sus tests, y al final el README). Cuando todo estaba terminado y el CI en verde, abrí una pull request a `main` (la #5) y la fusioné.

Una cosa que me pasó: hice el commit del README sin haber puesto la fecha de descarga del dataset. Como ya lo había subido, no reescribí la historia; hice un commit nuevo ("Añadir la fecha de descarga del dataset al README que me faltaba") y el CI volvió a salir en verde.

## 8. Cosas que mejoraría

- En `ventas_app` está la función `top3_prod_porimporte`, pero `cli.py` solo muestra el total por región. Se podría imprimir también el top 3.
- La clase `ValidationError` de `ventas_app/validator.py` no se usa.
- Los bucles y las reglas están escritos de forma simple a propósito; se podrían hacer más cortos, pero así me resulta más fácil leerlos y explicarlos.

## Anexo: código de los tests

Aquí está el código completo de los tests, tal y como está en el repositorio.

### Parte A: tests/ventas

*proyecto/tests/ventas/conftest.py*

```
"""Fixtures compartidas por los tests de ventas_app."""

import pandas as pd
import pytest


@pytest.fixture
def ventas_mini():
    """DataFrame pequeño en memoria: 1 fila válida y 2 inválidas."""
    return pd.DataFrame({
        "fecha": ["2026-01-01", "2026-01-02", "2026-01-03"],
        "region": ["Norte", "Sur", "Norte"],
        "producto": ["A", "B", "A"],
        "unidades": [2, None, 5],             # la fila 2 no tiene unidades -> inválida
        "precio_unitario": [10.0, 20.0, -1.0],  # la fila 3 tiene precio negativo -> inválida
        "cliente_id": ["C1", "C2", "C1"],
    })
```

*proyecto/tests/ventas/test_loader.py*

```
"""Tests de loader.py (E3 Parte 2: errores)."""

import pytest

from ventas_app.loader import DataLoadError, load


def test_load_lee_un_csv(tmp_path):
    # tmp_path es una carpeta temporal que pytest crea y borra sola
    ruta = tmp_path / "ventas.csv"
    ruta.write_text("region,unidades\nNorte,2\nSur,3\n", encoding="utf-8")

    frame = load(ruta)

    assert len(frame) == 2
    assert list(frame.columns) == ["region", "unidades"]


def test_load_con_path_inexistente_lanza_error(tmp_path):
    # Esperamos que falle con nuestra excepción de dominio
    with pytest.raises(DataLoadError):
        load(tmp_path / "no_existe.csv")
```

*proyecto/tests/ventas/test_validator.py*

```
"""Tests de validator.py (E3 Partes 1 y 2)."""

import pandas as pd
import pytest

from ventas_app.validator import validar_ventas


def test_separa_validos_e_invalidos(ventas_mini):
    # Act: llamamos a la función con los datos de la fixture
    validos, errores = validar_ventas(ventas_mini)
    # Assert: de las 3 filas, 1 es válida y 2 son inválidas
    assert len(validos) == 1
    assert len(errores) == 2


def test_calcula_el_importe(ventas_mini):
    validos, _ = validar_ventas(ventas_mini)
```

```
    # La fila válida tiene 2 unidades a 10.0 euros -> 20.0
    assert validos["importe"].iloc[0] == 20.0


def test_no_modifica_el_dataframe_original(ventas_mini):
    validar_ventas(ventas_mini)
    # La función trabaja sobre una copia, así que el original no gana la columna importe
    assert "importe" not in ventas_mini.columns


@pytest.mark.parametrize(
    ("precio", "es_valido"),
    [
        (0, False),   # precio cero -> inválido
        (-1, False),  # precio negativo -> inválido
        (1, True),    # precio positivo -> válido
    ],
)
def test_precios_en_los_bordes(precio, es_valido):
    # Una sola fila: lo único que cambia entre casos es el precio
    frame = pd.DataFrame({"unidades": [3], "precio_unitario": [precio]})
    validos, _ = validar_ventas(frame)
    assert (len(validos) == 1) == es_valido
```

*proyecto/tests/ventas/test_metrics.py*

```
"""Tests de metrics.py."""

import pandas as pd

from ventas_app.metrics import top3_prod_porimporte, total_por_region


def test_total_por_region():
    validos = pd.DataFrame({
        "region": ["Norte", "Sur", "Norte"],
        "importe": [20.0, 30.0, 40.0],
    })

    totales = total_por_region(validos)

    assert totales["Norte"] == 60.0  # 20 + 40
    assert totales["Sur"] == 30.0
    assert totales.index[0] == "Norte"  # viene ordenado de mayor a menor


def test_top3_productos_por_importe():
    validos = pd.DataFrame({
        "producto": ["A", "B", "C", "D"],
        "importe": [10.0, 40.0, 30.0, 20.0],
    })

    top = top3_prod_porimporte(validos)

    assert len(top) == 3  # solo los 3 mejores
    assert list(top.index) == ["B", "C", "D"]  # de mayor a menor importe
```

*proyecto/tests/ventas/test_cli.py*

```
"""Tests de cli.py (smoke test: comprobamos que el programa corre de principio a fin)."""

import sys

import pandas as pd

from ventas_app.cli import main


def test_main_genera_el_csv_de_salida(tmp_path, monkeypatch, capsys, ventas_mini):
    entrada = tmp_path / "entrada.csv"
    salida = tmp_path / "salida.csv"
    ventas_mini.to_csv(entrada, index=False)
```

```
    # Simulamos que el usuario escribe: --input ... --output ...
    monkeypatch.setattr(sys, "argv", ["cli", "--input", str(entrada), "--output", str(salida)])

    main()

    # Se ha creado el CSV de salida y solo contiene la fila válida
    assert salida.exists()
    assert len(pd.read_csv(salida)) == 1
    # Y por pantalla se imprime el resumen
    assert "Hay 1 válidos y 2 invalidos" in capsys.readouterr().out
```

*proyecto/tests/ventas/test_integracion.py*

```
"""Test de integración con el CSV real del curso (E3 Parte 3)."""

from pathlib import Path

import pytest

from ventas_app.loader import load
from ventas_app.validator import validar_ventas

# proyecto/data/ventas.csv, calculado desde la ubicación de este fichero
CSV_CURSO = Path(__file__).resolve().parents[2] / "data" / "ventas.csv"


@pytest.mark.integration
def test_csv_curso_tiene_140_validas_y_10_invalidas():
    frame = load(CSV_CURSO)

    validos, errores = validar_ventas(frame)

    assert len(validos) == 140
    assert len(errores) == 10
```

## Parte B: tests/internet

*proyecto/tests/internet/conftest.py*

```
"""Fixtures compartidas por los tests de internet_app."""

import pandas as pd
import pytest


@pytest.fixture
def sueno_mini():
    """DataFrame pequeño en memoria: 1 fila válida (U1) y 3 inválidas (U2, U3 y U4)."""
    return pd.DataFrame({
        "user_id": ["U1", "U2", "U3", "U4"],
        "age": [30, 12, 25, 40],                        # U2 tiene 12 años -> inválida
        "gender": ["Male", "Female", "Non-Binary", "Female"],
        "occupation_type": ["Student", "Remote Tech", "Corporate 9-to-5", "Student"],
        "chronotype": ["Night Owl", "Morning Lark", "Intermediate", "Intermediate"],
        "bedtime_phone_minutes": [90, 30, 45, 120],
        "primary_bedtime_app": ["TikTok / Reels", "YouTube", "YouTube", "TikTok / Reels"],
        "screen_brightness_pct": [60, 50, 40, 70],
        "blue_light_filter_active": [1, 0, 2, 1],      # U3 tiene un 2 (solo vale 0 o 1) ->
            inválida
        "caffeine_post_5pm_mg": [0, 50, 100, 0],
        "physical_activity_min": [30, 20, 40, 10],
        "sleep_latency_min": [30.0, 20.0, 25.0, 60.0],
        "total_sleep_hours": [5.0, 8.0, 7.0, 4.0],
        "deep_sleep_pct": [20.0, 22.0, 21.0, 19.0],
        "rem_sleep_pct": [20.0, 19.0, 18.0, 17.0],
        "morning_alarm_snoozes": [3, 0, 1, 5],
        "next_day_fatigue_score": [7.0, 1.5, 3.0, 9.0],
        "sleep_debt_category": ["Moderate Debt", "Optimal Recovery", "Mild Deficit", "Otra"],  #
            U4: "Otra" no existe -> inválida
    })
```

*proyecto/tests/internet/test_loader.py*

```
"""Tests de loader.py."""

import pytest

from internet_app.loader import DataLoadError, load


def test_load_lee_un_csv(tmp_path, sueno_mini):
    # tmp_path es una carpeta temporal que pytest crea y borra sola
    ruta = tmp_path / "sueno.csv"
    sueno_mini.to_csv(ruta, index=False)

    frame = load(ruta)

    assert len(frame) == 4


def test_load_con_path_inexistente_lanza_error(tmp_path):
    with pytest.raises(DataLoadError):
        load(tmp_path / "no_existe.csv")


def test_load_con_columnas_que_faltan_lanza_error(tmp_path, sueno_mini):
    ruta = tmp_path / "incompleto.csv"
    sueno_mini.drop(columns=["age"]).to_csv(ruta, index=False)  # quitamos la columna age

    # match comprueba que el mensaje de error menciona la columna que falta
    with pytest.raises(DataLoadError, match="age"):
        load(ruta)
```

*proyecto/tests/internet/test_validator.py*

```
"""Tests de validator.py."""

import pandas as pd
import pytest

from internet_app.validator import validar_sueno


def test_separa_validos_e_invalidos(sueno_mini):
    validos, errores = validar_sueno(sueno_mini)
    # De las 4 filas, solo U1 es válida
    assert len(validos) == 1
    assert len(errores) == 3
    assert list(validos["user_id"]) == ["U1"]


def test_anade_la_columna_movil_alto(sueno_mini):
    validos, _ = validar_sueno(sueno_mini)
    # U1 usa el móvil 90 minutos, que son 60 o más -> movil_alto es verdadero
    assert validos["movil_alto"].iloc[0]


def test_no_modifica_el_dataframe_original(sueno_mini):
    validar_sueno(sueno_mini)
    # Trabaja sobre una copia, así que el original no gana la columna movil_alto
    assert "movil_alto" not in sueno_mini.columns


@pytest.mark.parametrize(
    ("edad", "es_valida"),
    [
        (17, False),  # menor de 18 -> inválida
        (18, True),   # justo el mínimo -> válida
        (65, True),   # justo el máximo -> válida
        (66, False),  # mayor de 65 -> inválida
    ],
)
```

```
def test_edad_en_los_bordes(sueno_mini, edad, es_valida):
    fila = sueno_mini.head(1).copy()  # la fila U1, que es válida
    fila["age"] = edad
    validos, _ = validar_sueno(fila)
    assert (len(validos) == 1) == es_valida


def test_un_valor_que_no_es_numero_es_invalido(sueno_mini):
    fila = sueno_mini.head(1).copy()
    fila["total_sleep_hours"] = "abc"  # no se puede convertir a número
    validos, errores = validar_sueno(fila)
    assert len(validos) == 0
    assert len(errores) == 1


def test_un_id_repetido_es_invalido(sueno_mini):
    fila = sueno_mini.head(1)
    repetida = pd.concat([fila, fila])  # la misma fila dos veces
    validos, errores = validar_sueno(repetida)
    # La primera se conserva y la repetida es un error
    assert len(validos) == 1
    assert len(errores) == 1


def test_los_porcentajes_de_sueno_no_pasan_de_100(sueno_mini):
    fila = sueno_mini.head(1).copy()
    fila["deep_sleep_pct"] = 70.0
    fila["rem_sleep_pct"] = 50.0  # 70 + 50 = 120 -> inválida
    validos, _ = validar_sueno(fila)
    assert len(validos) == 0
```

*proyecto/tests/internet/test_metrics.py*

```
"""Tests de metrics.py."""

import pandas as pd
import pytest

from internet_app.metrics import (
    correlacion_movil_sueno,
    fatiga_media_por_categoria,
    sueno_medio_por_app,
    sueno_medio_por_uso_movil,
)


@pytest.fixture
def validos_mini():
    """Seis personas ya validadas, con números fáciles para calcular a mano."""
    return pd.DataFrame({
        "bedtime_phone_minutes": [20, 30, 90, 120, 45, 150],
        "movil_alto": [False, False, True, True, False, True],
        "total_sleep_hours": [8.0, 7.0, 5.0, 4.0, 7.5, 3.5],
        "next_day_fatigue_score": [1.0, 2.0, 7.0, 9.0, 2.0, 10.0],
        "sleep_debt_category": [
            "Optimal Recovery", "Mild Deficit", "Moderate Debt",
            "Severe Sleep Debt", "Mild Deficit", "Severe Sleep Debt",
        ],
        "primary_bedtime_app": [
            "News / Reading", "YouTube", "TikTok / Reels",
            "TikTok / Reels", "YouTube", "TikTok / Reels",
        ],
    })


def test_sueno_medio_por_uso_movil(validos_mini):
    resultado = sueno_medio_por_uso_movil(validos_mini)

    assert resultado[False] == 7.5                          # (8 + 7 + 7.5) / 3
    assert resultado[True] == pytest.approx(4.17, abs=0.01)  # (5 + 4 + 3.5) / 3 = 4.1666...
```

```
def test_fatiga_media_por_categoria(validos_mini):
    resultado = fatiga_media_por_categoria(validos_mini)

    assert resultado["Severe Sleep Debt"] == 9.5       # (9 + 10) / 2
    assert resultado["Mild Deficit"] == 2.0            # (2 + 2) / 2
    assert resultado.index[0] == "Severe Sleep Debt"   # va primero la de más fatiga


def test_sueno_medio_por_app(validos_mini):
    resultado = sueno_medio_por_app(validos_mini)

    assert resultado["YouTube"] == 7.25                # (7 + 7.5) / 2
    assert resultado.index[0] == "TikTok / Reels"      # va primera la app con menos sueño


def test_correlacion_movil_sueno_es_negativa(validos_mini):
    resultado = correlacion_movil_sueno(validos_mini)

    # Más minutos de móvil, menos horas de sueño: la correlación es casi -1
    assert resultado == pytest.approx(-0.98, abs=0.01)
```

*proyecto/tests/internet/test_cli.py*

```
"""Tests de cli.py (smoke test: comprobamos que el programa corre de principio a fin)."""

import sys

import pandas as pd

from internet_app.cli import main


def test_main_genera_el_csv_de_salida(tmp_path, monkeypatch, capsys, sueno_mini):
    entrada = tmp_path / "entrada.csv"
    salida = tmp_path / "salida.csv"
    sueno_mini.to_csv(entrada, index=False)

    # Simulamos que el usuario escribe: --input ... --output ...
    monkeypatch.setattr(sys, "argv", ["cli", "--input", str(entrada), "--output", str(salida)])

    main()

    # Se ha creado el CSV de salida y solo contiene la fila válida
    assert salida.exists()
    assert len(pd.read_csv(salida)) == 1
    # Y por pantalla se imprime el resumen
    assert "Hay 1 válidos y 3 inválidos" in capsys.readouterr().out
```

*proyecto/tests/internet/test_integracion.py*

```
"""Test de integración con el CSV real del dataset de Kaggle."""

from pathlib import Path

import pytest

from internet_app.loader import load
from internet_app.validator import validar_sueno

# proyecto/data/bedtime_screentime_sleep_debt.csv, calculado desde la ubicación de este fichero
CSV_SUENO = Path(__file__).resolve().parents[2] / "data" / "bedtime_screentime_sleep_debt.csv"


@pytest.mark.integration
def test_csv_real_tiene_8500_validas_y_ninguna_invalida():
    frame = load(CSV_SUENO)

    validos, errores = validar_sueno(frame)

    assert len(validos) == 8500
    assert len(errores) == 0
```